
Course Project Report

Comparison of Artificial Neural Network Controller and PID Controller in Real Time Industrial Temperature Process Control System

Based on the paper by:

International Journal of Control Systems and Robotics, ISSN: 2367-8917

Submitted by:

Stuti Yadav

Roll No: B23CH1044

Department of Chemical Engineering

Submitted to: Prof. Varanasi Santhosh Kumar

April 13, 2026

0. Contents

1	Introduction	2
2	Limitations of the PID Controller	2
3	System Setup and Block Diagram	3
4	PID Controller Tuning using Ziegler–Nichols Method	3
5	Artificial Neural Network Controller	4
6	Working Principle of the Overall System	5
7	Experimental Results and Observations	5
7.1	Experiment 1: Setpoint Tracking	5
7.1.1	Setpoint Tracking Observations	6
7.2	Experiment 2: Variable Dead Time	7
7.2.1	Variable Dead Time Observations	8
7.3	Summary of Baseline Results	8
7.4	Why Does the ANN Perform Better?	8
8	Proposed Improvements to the ANN Controller	9
8.1	Improvement 1: Residual ANN	9
8.2	Improvement 2: LSTM-based Controller	10
8.3	Why Do We Consider Residual ANN and LSTM as Improvements? . . .	11
9	Conclusion	12

1. Introduction

This paper compares a PID (Proportional Integral Derivative) controller and an Artificial Neural Network (ANN) controller for real-time industrial temperature control. While PID is widely used due to its simplicity, it struggles with nonlinear systems and changing conditions. An ANN, by contrast, learns from data and adapts itself, making it more flexible. The paper trains a neural network to replicate and improve upon PID behaviour, testing both on a LabVOLT Model 3504 temperature setup to determine which performs better.

2. Limitations of the PID Controller

PID controllers have several key limitations, particularly in nonlinear or time-varying systems:

- **Overshoot and Undershoot:** Temperature often exceeds or falls below the setpoint before stabilising.
- **Slow Response:** PID takes longer to settle within the desired range.
- **Requires Re-tuning:** Cannot adapt automatically when operating conditions change.
- **Poor Disturbance Recovery:** Takes considerable time to return to setpoint after a disturbance.

3. System Setup and Block Diagram

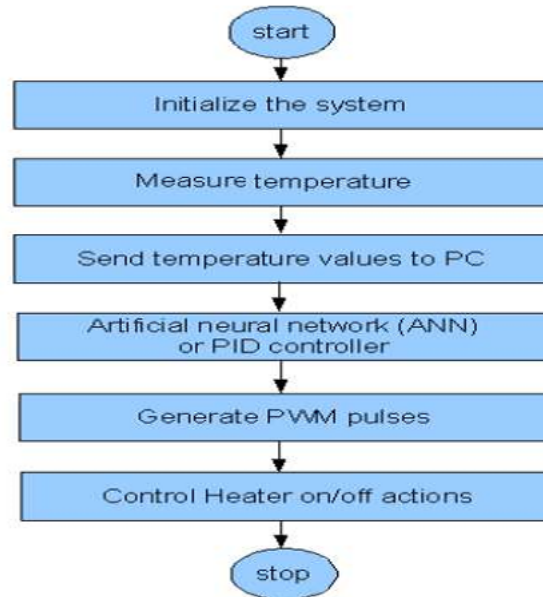


Figure 1: The flowchart of the system

The oven temperature is measured and sent to the controller (ANN or PID). Based on the error between the desired and actual temperature, a PWM signal is generated to adjust the heating element. This loop continues until the desired temperature is reached.

4. PID Controller Tuning using Ziegler–Nichols Method

The PID controller used in this experiment is tuned using the Ziegler–Nichols method. This is a standard technique where you first find two critical values: the critical gain (K_{cr}) and the critical time period (T_{cr}).

The general equation of a PID controller in time domain is:

$$u(t) = K_p e(t) + \frac{K_p}{T_i} \int_0^t e(t) dt + K_p T_d \frac{de(t)}{dt} \quad (1)$$

Where $e(t)$ is the error signal (difference between desired and actual temperature), K_p is the proportional gain, T_i is the integral time, and T_d is the derivative time.

In the frequency domain, this becomes:

$$U(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right) E(s) \quad (2)$$

The Ziegler–Nichols tuning method provides values for K_p , T_i , and T_d based on K_{cr} and T_{cr} as shown in the table below:

Table 1: Ziegler–Nichols Tuning Rules

Controller Type	K_p	T_i	T_d
P	$0.5 K_{cr}$	∞	0
PI	$0.45 K_{cr}$	$\frac{1}{1.2} T_{cr}$	0
PID	$0.6 K_{cr}$	$0.5 T_{cr}$	$0.125 T_{cr}$

After applying the Ziegler–Nichols formula, the PID transfer function becomes:

$$\frac{U(s)}{E(s)} = 0.075 K_{cr} T_{cr} \cdot \frac{\left(s + \frac{4}{T_{cr}} \right)}{s} \quad (3)$$

From the step response experiment done in the lab, the following values were calculated:

- Process Gain = 1.768
- Process Dead Time (τ_d) = 39.18 seconds
- Process Time Constant (τ) = 1303 seconds
- Ratio $\tau/\tau_d = 33.2$ (greater than 10, so the process is easy to control)

The final transfer function of the system came out to be:

$$G(s) = \frac{1.768}{1303s + 1} \times e^{-39.18s} \quad (4)$$

5. Artificial Neural Network Controller

A feedforward backpropagation ANN is used, where signals travel forward through the network and errors are propagated backward during training to adjust weights. The network is trained on PID-generated input-output data until the error falls below a threshold or a maximum number of epochs is reached, after which it directly replaces the PID as the controller.

Unlike PID, the ANN does not require re-tuning when operating conditions change — it generalises from its training data and adapts its control action based on the current input state, making it better suited for nonlinear and time-varying processes.

6. Working Principle of the Overall System

The oven heats up and its temperature is measured by a thermocouple, which sends the signal to a PC via Arduino. In MATLAB, the controller (ANN or PID) computes a control action based on the error between the setpoint and measured temperature. This is converted to a PWM signal by Arduino, which drives the TRIAC to adjust power to the heating element. The loop runs continuously until the measured temperature matches the setpoint. Both controllers are tested separately and their results compared.

7. Experimental Results and Observations

Two experiments were conducted to evaluate the performance of the PID and ANN controllers: a setpoint tracking test and a variable dead time test. Let us take a look.

7.1 Experiment 1: Setpoint Tracking

In this experiment, the simulation was run over a time horizon of 2000 time steps using the plant model $G(s) = \frac{1.768}{1303s+1}$ with a fixed process dead time of 40 steps. The system was evaluated at two distinct setpoints to observe and compare the transient and steady-state behaviour of both controllers across different reference values: the output was held at 0 for the first 400 steps, stepped up to 0.5 at $t = 400$, and then stepped up again to 1.0 at $t = 1400$. The ANN was trained on a richer multi-step profile (setpoints of 0.5, 0.8, 0.3, and 1.0) to expose it to a wide range of operating conditions before being evaluated on this test sequence. Figure 2 illustrates the temperature response of both controllers during this test.

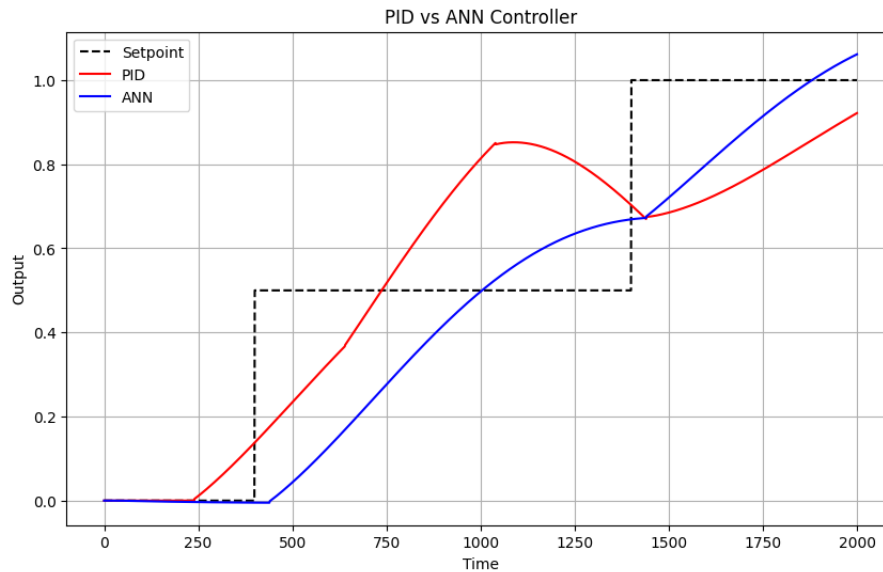


Figure 2: Setpoint tracking performance comparison between PID and ANN controllers.

Table 2 summarizes the quantitative performance metrics obtained from the setpoint tracking experiment.

Table 2: Performance Metrics – Setpoint Tracking Test

Metric	PID	ANN
MSE	0.04980	0.04171
RMSE	0.22315	0.20422
MAE	0.18933	0.14541
Steady-State Error	0.07829	0.06155
Overshoot	-0.07829	0.06155

7.1.1 Setpoint Tracking Observations

- The ANN was trained on PID-generated control data using a 9-feature input vector consisting of the current and three previous error values, current and three previous plant outputs, and the running integral of the error — capturing both the history and cumulative behaviour of the process.
- The ANN controller achieved a lower MSE (0.04171 vs 0.04980) compared to the PID, indicating closer tracking of the desired setpoint across both step changes.
- The RMSE and MAE values confirm that the ANN exhibited smaller average deviations from the reference, with MAE reducing from 0.18933 to 0.14541.

- The steady-state error of the ANN (0.06155) was lower than that of the PID (0.07829), reflecting better long-term accuracy after each step change.
- The ANN demonstrated more controlled transient behaviour owing to its ability to generalise from the multi-setpoint training data.

7.2 Experiment 2: Variable Dead Time

This experiment evaluates how well each controller handles sudden changes in the process dead time — a scenario common in real industrial systems due to varying transport delays. Both controllers were tested on a constant setpoint of 1.0 over 2000 time steps, but with the dead time changed across three distinct phases: 20 steps for $t < 800$, 60 steps for $800 \leq t < 1400$, and 30 steps for $t \geq 1400$. This sudden tripling of dead time in the middle phase (from 20 to 60 steps) represents a severe change in process dynamics. The PID was tuned for the nominal dead time of 40 steps and could not adapt, while the ANN received the same 9-feature input vector as before and relied on its learned generalisation capability.

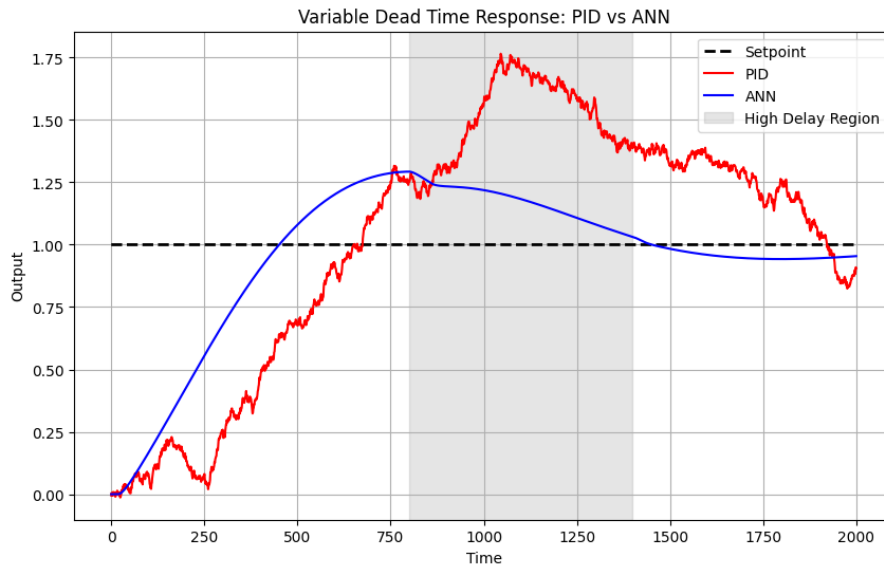


Figure 3: Variable dead time performance comparison between PID and ANN controllers.

Table 3 presents the performance metrics recorded during the variable dead time experiment.

Table 3: Performance Metrics – Variable Dead Time Test

Metric	PID	ANN
MSE	0.26552	0.10113
RMSE	0.51529	0.31800
MAE	0.44023	0.21385
Oscillation Index	16.02267	1.65524
Delay Region Error	0.50782	0.16852

7.2.1 Variable Dead Time Observations

- The PID controller, tuned for a fixed dead time of 40 steps, degraded significantly when the dead time jumped to 60 steps during the middle phase ($800 \leq t < 1400$). Its oscillation index of 16.02 versus the ANN's 1.66 quantifies how aggressively the PID output oscillated during this phase.
- The Delay Region Error — defined as the mean absolute error specifically during the high-delay phase ($800 \leq t < 1400$) — was 0.50782 for PID compared to just 0.16852 for ANN, showing the ANN was three times more accurate precisely when the dead time was most mismatched.
- The ANN controller maintained stable and relatively accurate control across all three dead time phases without any re-tuning, demonstrating strong generalisation to process dynamics outside its training conditions.
- All error metrics (MSE, RMSE, MAE) were more than halved for the ANN compared to PID, confirming its robustness to dead time variations.

7.3 Summary of Baseline Results

Across both experiments, the ANN controller outperformed the PID controller in all evaluated metrics.

7.4 Why Does the ANN Perform Better?

The ANN performs better than the PID controller in both setpoint tracking and variable dead time conditions because it learns the system behavior from data rather than relying on fixed control parameters.

In setpoint tracking, the ANN is able to predict more suitable control actions for different operating conditions since it has been trained on multiple setpoints. Unlike the PID

controller, which reacts only to the current error, the ANN uses both current and past values of error and output, resulting in smoother and more accurate responses.

In the case of variable dead time, the ANN is able to handle changes in system dynamics because it generalises from its training data. The PID controller, on the other hand, is tuned for a fixed dead time and cannot adapt to sudden changes, which leads to oscillations and higher errors.

Additionally, by using past information as input, the ANN can indirectly capture the effect of delays in the system, allowing it to respond more effectively under both steady and changing conditions.

8. Proposed Improvements to the ANN Controller

While the standard feedforward ANN performed better than the PID controller in my study, I observed that there was still scope for improvement. To further enhance control performance, I explored two advanced architectures: a Residual ANN and a Long Short-Term Memory (LSTM) network. Both were implemented and evaluated against the baseline ANN using the same experimental setup.

8.1 Improvement 1: Residual ANN

A Residual ANN introduces shortcut (skip) connections that add the input of a layer directly to its output, allowing the network to learn only the residual correction rather than the full mapping — this helps mitigate the vanishing gradient problem and leads to faster, more stable convergence during training.

In this implementation, the Residual ANN was used as a hybrid controller rather than just a modified ANN. Instead of predicting the full control output, it was trained to learn only the *residual error* between the actual PID signal and an estimated PID signal, i.e. $u_{\text{residual}}(t) = u_{\text{PID,actual}}(t) - u_{\text{PID,estimate}}(t)$. During testing, the final control action was calculated as $u = u_{\text{PID}} + \lambda \cdot u_{\text{residual}}$, where λ is an adaptive scaling factor.

Figure 4 shows the performance comparison between the standard ANN and the Residual ANN controller.

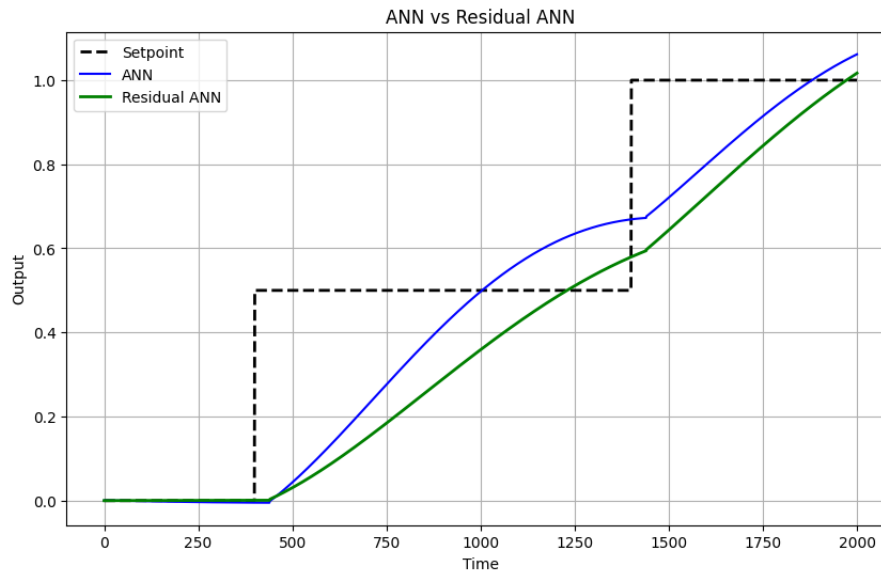


Figure 4: Residual ANN vs ANN – Performance Graph

Table 4 presents the performance metrics comparing the standard ANN and the Residual ANN.

Table 4: Performance Metrics – Standard ANN vs Residual ANN

Metric	ANN	Residual ANN
Steady-State Error	0.05028	0.00194
Overshoot	0.06155	0.01679

The Residual ANN achieved a drastic reduction in steady-state error from 0.05028 to just 0.00194 — a 96% improvement — and overshoot dropped from 0.06155 to 0.01679. This shows that adding small corrections to a stable PID output is an effective way to improve accuracy.

8.2 Improvement 2: LSTM-based Controller

A Long Short-Term Memory (LSTM) network is a type of recurrent neural network that uses gating mechanisms (input, forget, and output gates) to selectively remember or forget past information — making it well-suited for time-series control tasks where the current output depends on the history of past process states.

The LSTM model consisted of two stacked LSTM layers (64 and 32 units) followed by two Dense layers, and was trained on sequences of length 6. Each timestep included three features: the error $e(t)$, the plant output $y(t)$, and the running integral of the error. This

allowed the LSTM to use more past information compared to the standard ANN, which only uses a fixed 4-step history. During testing, the LSTM output was combined with a PID baseline to generate the final control action.

Figure 5 shows the performance comparison between the standard ANN and the LSTM controller.

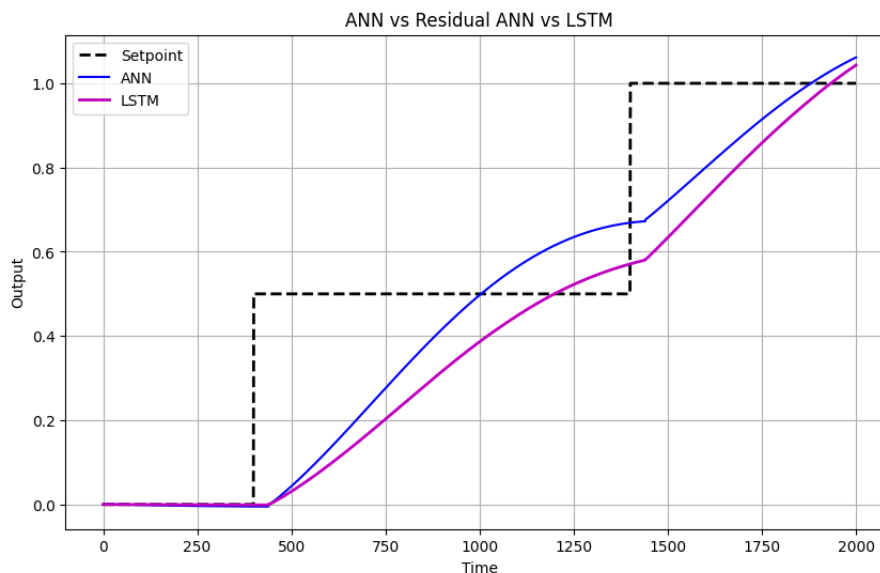


Figure 5: Performance comparison between the standard ANN and LSTM controllers.

Table 5 presents the performance metrics comparing the standard ANN and the LSTM controller.

Table 5: Performance Metrics – Standard ANN vs LSTM

Metric	ANN	LSTM
Steady-State Error (SSE)	0.05028	0.02786
Overshoot	0.06155	0.04310

The LSTM reduced the steady-state error to 0.02786 and overshoot to 0.04310 compared to the standard ANN, showing that using past data helps the controller handle system behavior better than a fixed input window.

8.3 Why Do We Consider Residual ANN and LSTM as Improvements?

Although the standard ANN performed better than the PID controller, it still has some important limitations.

First, the ANN learns a direct mapping from inputs to control output, which can be a difficult task. This may lead to small steady-state errors and less precise control. Second, it only uses a fixed number of past values as input, so it does not fully capture long-term system behavior. In real process control systems, the current output depends on a longer history, which the ANN cannot completely remember.

To overcome these limitations, we explore two improved architectures:

- **Residual ANN:** Instead of learning the full control action, the Residual ANN only learns the small correction (residual) over an already good baseline (PID). This makes the learning problem much easier and improves accuracy. As a result, it significantly reduces steady-state error and overshoot.
- **LSTM (Long Short-Term Memory):** The LSTM is designed to handle time-dependent data. Unlike ANN, it can remember information from many previous time steps using its internal memory. This helps it better understand system dynamics, anticipate changes, and improve control performance, especially in systems with delays or varying conditions.

Hence, both Residual ANN and LSTM directly address the key weaknesses of the standard ANN and lead to more accurate and robust control.

9. Conclusion

To conclude my report, this project implemented and compared a PID controller and a feedforward ANN controller for temperature control using a simulated plant model $G(s) = \frac{1.768}{1303s+1}$ with a 40-step dead time. The ANN was trained on PID-generated data using a 9-feature input vector that included current and past error, output, and integral values, and was tested on a two-step setpoint sequence (0.5 and 1.0).

In the setpoint tracking experiment, the ANN performed better than the PID in all metrics, reducing MSE from 0.04980 to 0.04171 and MAE from 0.18933 to 0.14541. In the variable dead time experiment — where the dead time changed between 20, 60, and 30 steps — the ANN performed much better. The oscillation index reduced from 16.02 to 1.66, and the delay region error reduced by about three times, showing that the ANN can handle changing system dynamics well.

Two improvements were then tested. The Residual ANN, which learns a correction over a PID baseline, gave the best improvement, reducing SSE by 96% from 0.05028 to 0.00194. The LSTM controller, which uses a 6-step sequence as input, reduced SSE to 0.02786 and overshoot to 0.04310. This demonstrates that modelling the time-series nature of

the control signal leads to measurably better transient behaviour.

In summary, the hierarchy of performance observed was: **Residual ANN > LSTM > Standard ANN > PID**. These findings confirm that for nonlinear, time-varying industrial control systems, hybrid and recurrent neural architectures offer significant and quantifiable improvements over both classical PID and standard feedforward ANNs.

Another thing I observed was that all results in this study were obtained on a synthetically generated dataset, simulated in Python using the plant transfer function from the paper with programmatically defined setpoint sequences — as the original experimental dataset from the LabVOLT setup was not available. Had the actual MATLAB/hardware data from the research paper been used, the differences between controllers would likely be more pronounced and the improvements from the Residual ANN and LSTM architectures even more clearly visible.